

CareerHub Pro — Technical Documentation

Comprehensive technical reference for developers working on or deploying CareerHub Pro.

Table of Contents

- 1. [Architecture Overview](#)
- 2. [Authentication & Authorization](#)
- 3. [Database Schema](#)
- 4. [API Reference](#)
- 5. [AI Integration Layer](#)
- 6. [File Storage \(S3\)](#)
- 7. [PDF Export System](#)
- 8. [Rate Limiting & Quotas](#)
- 9. [Input Validation](#)
- 10. [Frontend Architecture](#)
- 11. [Environment Variables](#)
- 12. [Deployment](#)
- 13. [Development Workflow](#)
- 14. [Troubleshooting](#)

1. Architecture Overview

Application Structure

CareerHub Pro uses the **Next.js 14 App Router** with a clear separation:

app/

|— (authenticated)/

|— api/

|— login/

|— signup/

|— onboarding/

|— page.tsx

← Protected routes, wrapped in AppShell (sidebar layout)

← REST API endpoints (all server-side)

← Public login page

← Public signup page

← Post-signup wizard

← Public landing page

Request Flow

Browser → Next.js Middleware (auth check)

→ App Router (page or API route)

→ Prisma ORM → PostgreSQL

→ AWS S3 (file operations)

→ LLM API (AI features)

Key Design Patterns

- **Route Groups:** `(authenticated)` group applies sidebar layout + auth guard to all career management pages.
- **Co-located Components:** Each page has a `_components/` directory for its client components.
- **Server-first:** Pages are server components by default; interactive parts are explicitly marked `"use client"`.
- **Shared Libraries:** All cross-cutting concerns (AI, auth, DB, S3, validation, rate-limiting) live in `lib/`.

2. Authentication & Authorization

Stack

- **NextAuth.js v4** with the **Credentials Provider** (email + bcrypt-hashed password)
- **Prisma Adapter** for session/account storage
- **JWT strategy** (stateless sessions)

Configuration — `lib/auth.ts`

```
session: { strategy: "jwt" },
callbacks: {
  jwt({ token, user }) { if (user) token.id = user.id; return token; },
  session({ session, token }) { session.user.id = token.id; return session; },
},
pages: { signIn: "/login" },
```

Route Protection

Middleware (`middleware.ts`):

- Protects: `/dashboard/*`, `/resumes/*`, `/jobs/*`, `/documents/*`, `/cover-letters/*`, `/ai-tools/*`
- Unauthenticated users → redirect to `/login`

API Routes:

- Every protected API route calls `getSession(authOptions)` independently
- Returns `401` if no session
- Derives `userId` from `session.user.id` — **never** from request parameters

User Registration — `POST /api/signup`

- Accepts `{ email, password, name }`
- Password hashed with `bcrypt` (12 rounds)
- Returns the created user (without password)

Type Extensions — `types/next-auth.d.ts`

Extends the NextAuth `Session` type to include `user.id: string`.

3. Database Schema

Provider

PostgreSQL via **Prisma ORM** (v6.7). Client generated to `node_modules/.prisma/client`.

Models

User

Core user record with onboarding state and career preferences.

Field	Type	Notes
<code>id</code>	String (CUID)	Primary key
<code>email</code>	String	Unique
<code>password</code>	String	bcrypt hash
<code>name</code>	String?	Display name
<code>onboarded</code>	Boolean	Onboarding completed flag
<code>careerMotivation</code>	String?	Onboarding answer
<code>jobTimeline</code>	String?	Onboarding answer
<code>helpArea</code>	String?	Onboarding answer
<code>targetJobTitles</code>	String	JSON array of titles (default <code>[]</code>)

AiUsage

Per-user, per-day, per-bucket usage counters for rate limiting.

Field	Type	Notes
<code>userId</code>	String	FK → User
<code>date</code>	String	YYYY-MM-DD (UTC)
<code>bucket</code>	String	<code>"ai"</code> or <code>"pdf"</code>
<code>count</code>	Int	Atomic increment via upsert
Unique		<code>(userId, date, bucket)</code>

Resume

Structured resume data. Arrays (experience, education, skills, certifications, projects, volunteering, awards, publications, interests) are stored as **JSON-stringified strings**.

Key fields: `title`, `template` (`"modern"` | `"classic"`), `fullName`, `email`, `phone`, `location`, `target-Title`, `summary`, plus the JSON array fields.

Job

Job application tracker with status workflow.

Field	Type	Notes
<code>status</code>	String	One of: <code>wishlist</code> , <code>applied</code> , <code>interview</code> , <code>offer</code> , <code>rejected</code>
<code>company</code>	String	Company name
<code>position</code>	String	Job title
<code>description</code>	String	Full job description text
<code>appliedDate</code>	DateTime?	When applied
Indexes		(<code>userId</code>), (<code>userId</code> , <code>status</code>)

Document

Cloud-stored files with metadata.

Field	Type	Notes
<code>cloudStoragePath</code>	String	S3 object key
<code>isPublic</code>	Boolean	Access control flag
<code>contentType</code>	String	MIME type
<code>fileSize</code>	Int	Bytes

CoverLetter

Cover letter content with company/role context.

Account / Session

Standard NextAuth adapter tables for OAuth accounts and sessions.

4. API Reference

All API routes live under `app/api/`. Protected routes return `401` without a valid session.

4.1 Resume Endpoints

GET /api/resumes — List all resumes for the authenticated user.

POST /api/resumes — Create or update a resume.

- Body: { id?, title, template, fullName, email, phone, location, targetTitle, summary, experience, education, skills, certifications, projects, volunteering, awards, publications, interests }
- If `id` is provided, updates the existing resume; otherwise creates a new one.

DELETE /api/resumes — Delete a resume.

- Body: { id: string }

4.2 Job Endpoints

GET /api/jobs — List all jobs for the authenticated user.

POST /api/jobs — Create or update a job application.

- Body: { id?, company, position, location?, url?, salary?, status?, notes?, description?, appliedDate? }

DELETE /api/jobs — Delete a job.

- Body: { id: string }

4.3 Cover Letter Endpoints

GET /api/cover-letters — List all cover letters.

POST /api/cover-letters — Create or update a cover letter.

- Body: { id?, title, content, jobTitle?, company? }

DELETE /api/cover-letters — Delete a cover letter.

- Body: { id: string }

4.4 Document Endpoints

GET /api/documents — List documents. Returns presigned download URLs.

POST /api/documents — Create document record after S3 upload.

- Body: { name, type, cloudStoragePath, contentType, fileSize, isPublic? }

DELETE /api/documents — Delete a document (record + S3 object).

- Body: { id: string }

4.5 Dashboard

GET /api/dashboard — Returns aggregated stats: resume count, job counts by status, recent activity, application timeline data.

4.6 Onboarding

GET /api/onboarding/status — Check if user has completed onboarding.

POST /api/onboarding — Save onboarding answers and mark user as onboarded.

- Body: { careerMotivation, jobTimeline, helpArea, targetJobTitles }

4.7 Upload

POST /api/upload/presigned — Generate S3 presigned PUT URL.

- Body: { fileName, contentType, isPublic? }
- Returns: { uploadUrl, cloudStoragePath, publicUrl? }

POST /api/upload/complete — Confirm upload (no-op placeholder for future validation).

4.8 AI Endpoints

All AI endpoints are POST-only, authenticated, rate-limited (60/day AI bucket), and return structured responses.

Streaming Endpoints (SSE / text)

POST /api/ai/suggestions — Resume improvement suggestions.

- Body: { resumeData: object }
- Returns: SSE stream of JSON suggestion objects

POST /api/ai/analyze-job — Job description analysis.

- Body: { jobDescription: string }
- Returns: SSE stream of structured analysis

POST /api/ai/cover-letter — Cover letter generation.

- Body: { resumeData: object, jobTitle: string, company: string, jobDescription?: string }
- Returns: Plain text stream

JSON Endpoints

POST /api/ai/ats-check — ATS compatibility scoring.

- Body: { resumeData: object, jobDescription?: string }
- Returns: Scored dimensions, keyword analysis, recommendations
- Two modes: generic (no JD) uses parseability focus; job-spec (with JD) adds keyword matching

POST /api/ai/job-match — Resume-job compatibility.

- Body: { resumeData: object, jobDescription: string }
- Returns: { matchScore, strengths[], gaps[], recommendations[] }

POST /api/ai/interview-prep — Interview preparation.

- Body: { resumeData: object, jobTitle: string, jobDescription?: string }
- Returns: Array of questions with suggested answers

POST /api/ai/tailor-cv — Tailored resume generation.

- Body: { resumeData: object, jobDescription: string }
- Returns: { tailoredResume: object, changesSummary: string[] }
- 90-second timeout for complex resumes

POST /api/ai/parse-cv — PDF resume parsing.

- Body: FormData with file (PDF) field
- Returns: Structured resume data extracted from the PDF
- 90-second timeout

POST /api/ai/parse-linkedin — LinkedIn profile parsing.

- Body: { profileText: string }
- Returns: Structured resume data

4.9 PDF Export

POST /api/resumes/export-pdf — Initiate async PDF export.

- Body: { resumeData: object }
- Rate-limited: 40/day (PDF bucket)
- Returns: { request_id: string, filename: string }

POST /api/resumes/export-pdf/status — Poll export status.

- Body: { request_id: string }

- Returns: { status: "SUCCESS"|"FAILED"|"PENDING", pdf?: string (base64) }

5. AI Integration Layer

Core Module — lib/ai.ts

All 9 AI API routes share this resilient LLM helper.

Exports

Export	Purpose
ChatMessage	Type: { role: string; content: any }
AIError	Error class with status and userMessage
extractJson<T>(raw)	Robust JSON extraction: strips code fences, walks balanced brackets, handles prose
callLLM(messages, opts?)	Non-streaming LLM call with timeout + exponential backoff retries
callLLMJson<T>(messages, opts?)	Like callLLM but parses JSON response + one corrective retry on malformed output
fetchLLMStream(messages, opts?)	Streaming LLM call with connection-level retry
aiErrorResponse(error)	Standard JSON error Response

Retry Logic

- **Retries on:** Network errors, timeouts, HTTP 429, HTTP 5xx
- **Does not retry on:** 400, 401, 403 (client errors)
- **Backoff:** Exponential with jitter
- **Default timeout:** 60 seconds (configurable per-call)
- **Streaming:** Retries the initial connection only; once headers arrive, the stream flows freely

JSON Extraction

The `extractJson<T>()` function handles common LLM response quirks:

1. Strips markdown code fences (````json ... ````)
2. Tries direct `JSON.parse` first
3. Falls back to balanced-bracket walking to find the first `{...}` or `[...]`
4. On `callLLMJson` failure, sends one corrective retry asking the LLM to output valid JSON

LLM Configuration

- **Endpoint:** Configured via `ABACUSAI_API_KEY` environment variable
- **Model:** `gpt-5.4-mini` (configurable)
- **Auth:** Bearer token

6. File Storage (S3)

Configuration — `lib/aws-config.ts`

AWS S3 client initialized from environment variables.

Upload Flow

1. Client requests a presigned URL via `POST /api/upload/presigned`
2. Client uploads directly to S3 using the presigned URL (PUT)
3. Client confirms via `POST /api/upload/complete`
4. Server creates a `Document` record with the `cloudStoragePath`

Download Flow

- `GET /api/documents` generates fresh presigned download URLs for each document
- URLs are short-lived (default expiry) and never stored in the database

Security

- All documents are **private by default** (`isPublic: false`)
- Only the owning user can access their documents (session check on every request)
- `cloudStoragePath` stored in DB; signed URLs generated on-demand

7. PDF Export System

Architecture

PDF export uses an external HTML-to-PDF conversion service (async).

Server Flow (`app/api/resumes/export-pdf/route.ts`)

1. Receives resume data
2. Renders HTML template (“modern” or “classic” style)
3. Submits HTML to the conversion service
4. Returns `{ request_id, filename }` immediately

Client Flow (`lib/pdf-export.ts`)

1. `exportResumePdf(resumeData, filename, onProgress?)` — main entry point
2. POSTs to `/api/resumes/export-pdf` → gets `request_id`
3. Polls `/api/resumes/export-pdf/status` every 1.5 seconds (max 120 attempts = 3 min)
4. On SUCCESS: decodes base64 PDF → triggers browser download
5. Calls `onProgress(percent, label)` at each step for UI updates

Templates

Two HTML/CSS templates rendered server-side:

- **Modern:** Teal accent color, clean sans-serif layout
 - **Classic:** Traditional serif typography, conservative styling
-

8. Rate Limiting & Quotas

Module — `lib/rate-limit.ts`

How It Works

1. Each AI or PDF request calls `enforceQuota(userId, bucket)`
2. Performs an **atomic upsert** on `AiUsage` table: increment count for `(userId, today, bucket)`
3. Returns `{ ok, used, limit, remaining }`
4. If `!ok`, the route returns a `429` response **before** making any paid API call

Configuration

```
export const DAILY_LIMITS: Record<string, number> = {
  ai: 60, // All LLM-backed endpoints
  pdf: 40, // PDF exports
};
```

Day Boundary

UTC-based (`YYYY-MM-DD`), so all users reset at the same time regardless of timezone.

Failure Mode

Fails open: If the database is unreachable during the quota check, the request is allowed through. This ensures a transient DB issue never blocks a paying user's core feature.

Integration

Every AI route has this pattern at the top:

```
const quota = await enforceQuota(session.user.id, 'ai');
if (!quota.ok) return quotaResponse('ai', quota.limit);
```

The PDF export route uses `bucket: 'pdf'`.

9. Input Validation

Module — `lib/validate.ts`

```
export async function readJson(request: Request): Promise<any | null>
export function isEmptyString(v: unknown): v is string
```

`readJson(request)`

- Calls `request.json()` inside a try/catch
- Returns the parsed body, or `null` if parsing fails (malformed JSON, empty body, etc.)
- Routes check `if (!body)` return `400` before accessing any fields

`isEmptyString(v)`

- Returns `true` only if `v` is a `string` AND `v.trim().length > 0`
- Used to validate text fields before passing to the LLM (prevents wasted API calls on empty/wrong-typed input)

Validation Pattern

All 9 AI routes follow this pattern:

```
const body = await readJson(request);
if (!body) return new Response(JSON.stringify({ error: 'Invalid or missing JSON body' }), { status: 400 });
if (!isEmptyString(body.jobDescription)) return new Response(JSON.stringify({ error: 'jobDescription is required' }), { status: 400 });
if (typeof body.resumeData !== 'object') return new Response(JSON.stringify({ error: 'resumeData object is required' }), { status: 400 });
```

10. Frontend Architecture

Layout Hierarchy

```
RootLayout (fonts, theme, providers)
├─ (authenticated)/layout.tsx
│   └─ AppShellWrapper (client)
│       └─ AppShell (sidebar nav + main content area)
│           └─ Page content
```

Sidebar Navigation

Defined in `components/layouts/app-shell.tsx` :

- Dashboard
- Resumes
- Jobs
- Cover Letters
- Documents
- AI Tools
- Interview Prep

State Management

- **Server State:** React Query (`@tanstack/react-query`) for API data fetching/caching
- **Client State:** Zustand / Jotai for UI state
- **Forms:** React Hook Form + Zod validation
- **Theme:** `next-themes` (light/dark)

Key Client Components

Component	Location	Purpose
ResumesContent	resumes/_components/	Resume list, CRUD, PDF export, AI tool integration
TailorCV	resumes/_components/	Two-step AI tailoring dialog (input → review/save)
ATSChecker	resumes/_components/	ATS scoring with optional job description
JobMatcher	resumes/_components/	Quick resume-job match scoring
ResumePreview	resumes/_components/	Live resume rendering (modern/classic templates)
JobsContent	jobs/_components/	Kanban-style job tracker
CoverLettersContent	cover-letters/_components/	Cover letter manager with AI generation
DocumentsContent	documents/_components/	File upload/download manager
DashboardContent	dashboard/_components/	Stats cards + activity chart
InterviewPrepContent	interview-prep/_components/	AI interview Q&A generator
ResumeSuggestions	ai-tools/_components/	Standalone AI suggestions tool
JobAnalysis	ai-tools/_components/	Standalone job analysis tool

Toast Notifications

Uses **Sonner** (`components/ui/sonner.tsx`) for all user feedback (success, error, loading/progress).

11. Environment Variables

Variable	Required	Description
<code>DATABASE_URL</code>	✓	PostgreSQL connection string
<code>NEXTAUTH_SECRET</code>	✓	JWT signing secret
<code>NEXTAUTH_URL</code>	✓	App base URL (used for auth callbacks)
<code>ABACUSAI_API_KEY</code>	✓	LLM API key (OpenAI-compatible)
<code>AWS_ACCESS_KEY_ID</code>	✓	AWS IAM access key
<code>AWS_SECRET_ACCESS_KEY</code>	✓	AWS IAM secret key
<code>AWS_REGION</code>	✓	AWS region (e.g., <code>us-east-1</code>)
<code>AWS_S3_BUCKET</code>	✓	S3 bucket name for document storage
<code>HTML2PDF_API_URL</code>	For PDF export	HTML-to-PDF service endpoint
<code>HTML2PDF_API_KEY</code>	For PDF export	HTML-to-PDF service API key

Security Notes:

- Never commit `.env` to version control
- `NEXTAUTH_SECRET` should be generated with `openssl rand -base64 32`
- All secrets are server-side only (never exposed to the browser)

12. Deployment

Build

```
npm run build
```

Produces a standalone output in `.build/standalone/` suitable for containerized deployment.

Production Requirements

- Node.js 18+ runtime
- PostgreSQL database (accessible from the deployment environment)
- AWS S3 bucket with appropriate IAM permissions
- Environment variables set (see §11)

Health Check

The landing page (/) and auth endpoints (/api/auth/session) can serve as health checks.

13. Development Workflow

Local Setup

```
# Install dependencies
npm install

# Generate Prisma client
npx prisma generate

# Push schema to dev database
npx prisma db push

# Seed sample data
npx prisma db seed

# Start dev server
npm run dev
```

Database Changes

1. Edit `prisma/schema.prisma`
2. Run `npx prisma db push` (for additive changes)
3. For breaking changes, use `npx prisma migrate dev`
4. Run `npx prisma generate` to update the client

Adding a New AI Feature

1. Create route in `app/api/ai/<feature>/route.ts`
2. Import from `lib/ai.ts` (`callLLMJson` for JSON, `fetchLLMStream` for streaming)
3. Add `enforceQuota` check at the top
4. Add input validation with `readJson` + `isNonEmptyString`
5. Create client component in the relevant `_components/` directory

Code Conventions

- `"use client"` only on interactive components
 - `export const dynamic = "force-dynamic"` on routes that read `process.env` at runtime
 - All API responses use `NextResponse.json()` or `new Response()`
 - Error responses always include `{ error: string }` JSON body
-

14. Troubleshooting

Common Issues

Issue	Cause	Fix
PrismaClientInitialization-Error	Missing DATABASE_URL	Ensure .env is loaded; use dotenv/config for scripts
401 on API routes	Session expired or missing	Check NEXTAUTH_SECRET matches; ensure cookies are sent
PDF export timeout	Slow HTML-to-PDF service	The async system polls for up to 3 minutes; check service status
S3 upload 403	Wrong IAM permissions	Verify PutObject + GetObject permissions on the bucket
AI 429 responses	Daily quota exceeded	Wait for midnight UTC reset, or adjust DAILY_LIMITS
Malformed AI response	LLM returned non-JSON	extractJson handles most cases; check lib/ai.ts logs
Hydration errors	Server/client mismatch	Ensure dates use SafeDate, browser APIs in useEffect only

Useful Commands

```
# Check database state
npx prisma studio

# Reset and re-seed (DESTRUCTIVE)
npx prisma db push --force-reset && npx prisma db seed

# Type check
npx tsc --noEmit

# View Prisma query logs
DEBUG="prisma:query" npm run dev
```